Human cross-verification of explanation reports: design

Design document, 2026-11-11, branch `design/human-verification`. Nothing here is implemented. It covers what a human reviewer needs to cross-verify an AI-drafted explanation report quickly and rigorously. The reviewer is Mike, or a clinical collaborator for the domain questions. "The explaining agent" is the AI agent that builds the annotation hierarchy and writes hypotheses and evidence. "The drafting agent" writes the plain-language prose. "The verifier" is an independent blind agent that follows the `explanation-verifier` skill.

Inputs:

- `docs/reviews/explanation-errors-2026-11.md` ("the review"): the root causes of E1 to E15;
- `experiments/errata/2026-11-11/ERRATA.md`;
- verifier-benchmark run 1 (`SCORE.md`, `ADJUDICATION.md`);
- the verifier and plain-language skills;
- `experiments/10seed/EXPLAIN_PROTOCOL.md` and `PREREG.md`;
- the verification pack (`explaneat/export/verification.py`, `experiments/10seed/verification/`);
- the Review page, `RecordPanel` and `ToolsDrawer`, and `docs/ui_audit.md`.

0. Summary

The problem. A reviewer today gets a 20 to 40 page certificate and a read-down document with about 50 plain sentences and about 185 numbers per genome (medians over the 80 drafted layers). On top of that come the technical hypotheses and evidence narratives. There is one yes/no verdict for the whole genome (`mike_verification`). Nothing tells the reviewer which of the numbers came from code and which were typed by hand. Nor does anything say which sentences a checker has already confirmed, or where the known error mechanisms are likely to have struck. The review found that the errors cluster in exactly these places:

- hand-typed numbers in free text (8 of the 15 errors);
- numbers retyped from claims (5 errors);
- scope words;
- formulas written as exact.

The design in one paragraph:

- Every number carries a link to the query that produced it, with a **reproduce** button.
- Every sentence gets a computed status badge: computed, rechecked, unchecked or failed.
- A risk score, built from the review's error mechanisms, ranks the sentences into a short review queue. The verifier's verdicts sit on the same sentences.
- The human records confirmed, wrong or unsure per sentence or claim. The verdicts live in a small append-only file keyed by stable sentence and claim ids, so they survive re-drafts and errata, and a diff shows what changed since the last review.
- A sign-off rule turns this into a defined "Mike-verified": review every gate item and every high-risk item, plus a seeded random sample of 10 of the rest. The rule gives a reportable upper bound on the residual error rate.

Must-haves:

1. sentence ids, status and the verdict store;
2. the risk queue in the verification pack;
3. the verifier overlay;
4. the sign-off command and the PREREG definition;
5. a minimal verdict list in the UI;
6. the provenance appendix and reproduce, once the integrity branch lands.

Effort: about 6 working days for the must-haves, and about 4 more for the nice-to-haves. **Mike's**

time: about 15 to 25 minutes per genome on records with provenance, and 30 to 45 minutes on the legacy records.

1. Status of the integrity branch, and assumptions

`fix/explanation-integrity` exists only as a local branch, at the same commit as `main` (8ab241a). Nothing had been committed or pushed by 2026-11-11 12:20 NZDT. This design therefore follows the review's recommendations (P1 and P2), and the assumptions below are marked **A1 to A7**. Section 8.3 says what changes if the branch differs.

#	Assumption about the integrity branch	Why this design needs it	If it differs
A1	Every query result is stored as a **computation record** with an id that is unique within the explanation (e.g. `q17`). It holds the tool, the full arguments (split, `where`, n, seed, `max_samples`), the value or values with their field names, the unit (z, raw, logit or probability), the scope (`direct`, `part:<ann>` or `whole_model`), the population label, the operation sequence number, a code hash and a data hash.	Provenance cards and reproduce are read from it.	Any record that holds the tool and the arguments is enough for reproduce. Scope and unit can be inferred from the tool where they are missing, and are then shown as "inferred".
A2	Text stores **references** such as `{q17.mean_risk}` in hypotheses, narratives and payloads, and slots such as `{C086.groups[1].to}` in drafts. Code renders them, rounding once. Both the source text (with the references) and the rendered text are kept.	Sentence ids are hashed from the source text, so a changed value keeps the id but changes the content hash (section 6).	If only rendered text is kept, the sentence id falls back to the location plus the text with numbers masked (section 6.1).
A3	**Write-time checks** (the free-text checker, the formula check and the number-word lint) return findings that are stored with the text field they checked: status, findings and code hash.	They feed the sentence status (section 3) and the gate "open write-time finding" (section 4).	Run `recheck` at export instead, which is today's behaviour.
A4	`amend_evidence` keeps the history: `evidence[j].history = [{at, was, now, reason}]`.	It feeds the "changed since you verified" diff.	Use the git history of the records and layers, as `prose_history.py` does.
A5	Code-filled drafts derive their citations from the slots.	A drafted sentence with only slots is "computed" by construction.	The drafted numbers are "rechecked" through `recheck`'s prose checks, as today.
A6	Claims keep their stable `key` (e.g. `ann_40:direction:ca`) across re-extraction, while the display ids (`C01`) may be renumbered.	Claim verdicts are keyed by `key` (this holds today).	Key by part, kind and variable instead.
A7	The verifier's location strings (skill section 6) are unchanged.	The verifier overlay joins on them.	Add a mapping table.

2. Traceability: every number links to its provenance

2.1 What a provenance card shows

A provenance card belongs to one rendered number. It shows these fields, taken from the computation record (A1), from the claim, or from `recheck`'s statement:

Field	Example (E1, heart-s5, after the fix)
Quantity	mean predicted risk (probability)
Population	training patients with 290 on, and 204_a and 204_b off
Where-expression	`n290 > 0 & n204a == 0 & n204b == 0`
Scope	whole model (the output), not this part's contribution
Unit, split, n	probability; train; n = 35
Value	0.181257 (rendered as "0.18")
Source	`q4`, `verify subgroup`; operation seq 23; code `a1c9e2`; data `5f03..`
Command	`uv run python -m explaneat verify subgroup heart-s5 --where "..."`
Reproduced	match (0.181257), 2026-11-14 10:02

A number can have one of three kinds of source:

- `query`: a computation record (A1);
- `claim`: a field of a plain-language claim, addressed by claim key and field path, e.g. `ann_31: direction:age / risk.to`;
- `freetext`: a number that only `recheck` parsed. The card then shows `recheck`'s statement (kind, node, stated value, model value) and says "parsed from the text, no stored query".

A number with none of these sources has no card. It is shown with a dotted underline and the tooltip "no provenance".

2.2 The reproduce action

Reproduce reruns the source on the current model and compares the result with the stored value.

Source	How it reruns	Match rule
`query`	Run the stored tool and arguments through the `explaneat verify` code on the genome, with its operations replayed (verify cache)	Counts: exact. Floats: relative difference below 1e-6, and the rendered text is unchanged
`claim`	`recheck`'s claim recomputation (cached by genome, operations, data and code), then that field	`recheck`'s own tolerance (`close()`), and the same rendered text
`freetext`	`recheck`'s free-text statement for that location	Its status (`ok` or `mismatch`)

The result is one of five outcomes:

Outcome	Meaning	Badge effect
`match`	The value reproduces under the current code and model.	none
`match-rounded`	The value differs, but the rendered text is the same.	none; the card says so

Outcome	Meaning	Badge effect
`drift`	The value is the same, but the code hash or the operations changed since the query.	none; the card says so
`mismatch`	The rendered text would change.	the sentence fails (section 3)
`unavailable`	The data is restricted or there is no database.	the sentence is unchecked

Speed. One query reruns in about 1 to 3 s on a warm verify cache, Adult included. Claims come from `recheck`'s cache.

Bulk run. `explaneat review reproduce <label> --all` reruns every source of a genome. It writes `review/<label>.reproduce.json` with a hash of its inputs, and sign-off (section 7) requires a full reproduce run with no mismatch.

2.3 Where provenance appears

UI (Statements tab, section 8.2):

- every rendered number is a link;
- clicking one opens its card, with **Reproduce** and **Copy command** buttons;
- an **Open in probe** button pre-fills the query's arguments where the tool is `probe` or `subgroup`;
- a sentence's badge popover lists the sources of all its numbers.

PDF. The export already has a review-marks switch: the `review_marks` argument of `render_review`, which sets `\{}reviewmarksfalse`. Both editions use it:

- **The review edition** (verification pack, marks on):
  - every sentence carries a small marginal tag with its sentence number and badge, e.g. "S14 C";
  - every number carries a superscript provenance id, e.g. "0.18^q4" or "about 27%^C83";
  - an appendix table "Provenance of the numbers", one row per id, has the columns: id, sentence, quantity, population, scope, unit, split, n, full-precision value, command, and the reproduce outcome at build time;
  - long commands wrap.
- **The reader edition** (marks off; a clinician's or examiner's copy) has:
  - no superscripts;
  - one closing line: "Every number in this document is linked to the computation that produced it; see the review edition or the JSON export."

JSON (export schema 1.2 to 1.3; additive, so older readers still work):

```
"provenance": {
  "queries":  [ {"id": "q4", "tool": "subgroup", "args": {...}, "values": {...},
                 "unit": "probability", "scope": "whole_model", "population": "...",
                 "n": 35, "split": "train", "op_seq": 23, "code": "a1c9e2", "data": "5f03.."} ],
  "claims":   [ {"key": "ann_31:direction:age", "display_id": "C30", "hash": "..."} ]
},
"text": {
  "sentences": [ {"sid": "s-9f2c41d0", "location": "operations.ischaemia_pathway.hypothesis[2]",
                  "source": "... mean predicted risk {q4.mean_risk} ...",
                  "rendered": "... mean predicted risk 0.18 ...",
                  "refs": [{"ref": "q4.mean_risk", "span": [41, 45], "kind": "query"}],
                  "status": "computed", "status_reasons": [],
                  "risk": {"score": 1.5, "tier": "C", "factors": ["prominence"]},
                  "verifier": [], "human": null} ]
}
```

3. Per-sentence status badges

3.1 Units

A **sentence** is the smallest unit that gets a badge, a risk score and a verdict. Sentences are taken from:

- every drafted text field: `lead`, `blocks`, `readdown.<part>.hypothesis` and `.evidence`, `worth_checking`, `summary` and `annotations`;
- every database text field: each annotation's `hypothesis`, each evidence `narrative`, and the payload's stated values, treated as one pseudo-sentence per payload;
- the record's `summary` and `resisted`.

Two rules govern the sentence splitter:

- it must split before a sentence that starts with a digit (the review's E2 note);
- it must not split inside a reference.

A **number atom** is one numeric token, or one **number word**, inside a sentence. The number words are: zero, none, no, all, every, only, never, always, exactly, entirely, sole(ly) and "the same". Words like "twice" and "half" are optional additions.

3.2 The badges

There are three badges and an override, as the brief asks. A sentence with no number atom and no shape or direction claim gets no badge (`n/a`).

Badge	Definition (all conditions must hold)
C: computed and auto-checked (green)	1. Every numeric atom is inside a reference (query or claim slot). 2. Every reference resolves. 3. Its last reproduce outcome is `match`, `match-rounded` or `drift`. 4. Every number word is either inside a reference or covered by a check: a reference whose query proves it, e.g. `enumerate` showing that no cell is non-zero, or a formula check. 5. Every cited claim is `verified`.
R: free text checked by recheck (amber)	Not C, and every numeric atom that is outside a reference was matched by `recheck` (free-text statement `ok`, or a prose number found in a cited claim). No atom is unmatched or unchecked. Number words outside references are allowed only if `recheck`'s direction or "only ever adds" check covered them.
U: unchecked; needs a human (grey)	At least one numeric atom or number word is neither referenced nor matched by `recheck`, or a reference could not be rerun (`unavailable`).
F: failed (red; the override)	Any reproduce `mismatch`, `recheck` mismatch, open write-time finding, or scope mismatch (section 4.2). It overrides C, R and U until it is resolved: fixed, or answered with a human verdict.

Precedence: F, then U, then R, then C. The badge's reasons list every atom that blocked a better badge, e.g. "'zero' is not referenced" or "'27' is in no claim".

C means "the number is what the stated query gives". It does **not** mean "the query is the right one for the words". That gap is exactly E8 and E9, which were correct whole-model numbers under a "direct" label. So a C sentence can still be wrong, and the risk score (section 4) and the random sample (section 7) are what look for referent errors.

3.3 How the status is computed and stored

- **Code:** a new `explaneat/review/status.py`. It takes:

  - the layer and the record;
  - the operations' text fields;
  - the computation records (A1);
  - the `recheck` report (`freetext.checked`, `unchecked_sentences`, `prose`, `findings`);
  - the last reproduce report.
- **When:**
  - on every draft store;
  - on `explaneat review status <label>`;
  - at the start of every export and verification pack.
- **Storage:** `experiments/<study>/review/<label>.status.json`. It is derived and can be regenerated, so it is not the source of truth. It holds, per `sid`: the badge, the reasons, the number atoms with their sources, the code hash and the time.
- **In the export:** the export copies the status into `text.sentences[].status`, and the verification pack prints the counts. For example: "C 112, R 31, U 18, F 0 (of 161 sentences with numbers)".
- **Headline metric** (the review's P3 asks for this): the U count, alongside `recheck`'s mismatches.

4. The risk-ranked review list

4.1 Factors and weights

Each factor comes from an error mechanism in the review. The weights are small integers, set so that the review's own errata would have landed where they belong (section 4.4). They are not fitted: with 15 errors there is nothing to fit.

Factor (once per sentence unless stated)	Mechanism and errata	Weight
Unchecked hand-typed number (U atom); per atom, at most 3	(f), (b) and (d) in free text: E1, E2, E5, E6, E8, E9, E11. About 1% of the numeric statements in the ten corrected genomes' database text were wrong (8 of about 770)	5
Retyped number checked only by `recheck` (R atom); per atom, at most 3	Referent ambiguity: 37 of `recheck`'s 47 false positives were wrong population, referent, quantity or configuration; also drafted retyping	2
Citation gap (`prose_uncited`)	(f) E4, E15	3
Scope or number word ("only", "all", "zero", "none", "never", "always", "every", "exactly") that is not proved by a reference	(e) E5 ("zero in area 415"); thal:7 "ONLY through these gates". Neither was reachable by `recheck`	4
Formula with "=" or "reduces to" that has not passed an exact write-time check	(e) E3; the five listed approximate formulas	4
"≈" formula with no tolerance stated	(e); review P3 ("≈" is otherwise a blanket escape)	2
Small group: a referenced or stated n below 30, or below 1% of training rows	E2 (26 rows), E5 (30 rows); `what_to_look_at`'s "rare regime" rule	2
Direct-or-whole-model ambiguity: an effect size for an input that also feeds a part, with no scope word, or with a scope word that differs from the query's scope	(d) E8, E9; the v5 "direct contribution only" rule	4
Named exception group from a generated shape claim (`status: mixed`, `exceptions[]`), or numbers for one population next to another population's label	(g) E7	2
Retyped percentage within 0.1 units of a rounding boundary (e.g. 27.4963)	(f) double rounding: E10, E12, E14	1

Factor (once per sentence unless stated)	Mechanism and errata	Weight
A number from another configuration or split (test AUC, linearised, ablated, "without X") with no label for it	the review's false-positive category D; the AAAI test AUCs	2
Verifier finding graded `weakened`	ADJUDICATION: most weakened findings on controls were genuine	2
Prominence multiplier: lead, summary, record `summary`, any hypothesis	Errors there propagate into copies (E8 and E9 reached the record summary)	x1.5

Gates. A gate puts an item into the queue regardless of its score:

- the sentence is F (section 3.2);
- the verifier `refuted` it;
- the sentence or claim **changed since the last human verdict** (section 6);
- a write-time finding is still open;
- a scope mismatch: the sentence says "direct", but its referenced query has `scope = whole_model`, or the reverse.

The scope mismatch can only be detected once A1 holds, and it turns E8 and E9 from judgement calls into automatic gates.

4.2 Score, tiers and ordering

```
score(s) = prominence(s) * sum_f weight_f * count_f(s)        (U and R atoms capped at 3 each)
tier(s)  = G  if any gate
           A  if score >= 5
           B  if 2 <= score < 5
           C  otherwise
order    = G (failed first, then verifier-refuted, then changed, then open finding),
           then A by score descending, then B, then C;
           ties in reading order (lead, blocks, root part down the hierarchy, technical text last)
```

Claims are scored too. A claim takes the highest score of the sentences that cite it, plus 2 if its `confidence.n_ok` is false, plus 2 if its shape `status` is `mixed`. Claims that no sentence cites are not queued: the prose is what gets read.

Budget cap. Legacy records have no references, so almost every number is a U atom: 45 to 62 unchecked numeric sentences per corrected genome (`recheck_coverage.json`). Reviewing all of tier A there would cost about 45 minutes per genome. So tier A is capped at the **top 25 by score**, and the overflow (tier A') is sampled (section 7). On records written with the integrity tools, tier A is expected to hold about 5 to 15 items.

4.3 Where it appears

- **CLI:** `explaneat review queue <label> [--tier G,A] [--json]`.
- **Verification pack:** a "Review queue" table replaces `what_to_look_at()`'s six bullets. It has one row per G and A item, with the columns: number, location, sentence (shortened), badge, factors, verifier grade, and a tick box. The structural "Recorded vs live" table stays as it is.
- **UI:** a "Queue" mode of the Statements tab (section 8.2).

4.4 Check against the errata

The errata as originally written (before any v4 tooling), scored by these rules:

Error	Factors	Score	Tier
E1, subgroup mean	U atom in a hypothesis; verifier refuted	7.5	G
E2, Thai “27 rows”	U atom; small n (26); hypothesis	10.5	A
E3, formula “=”	“=” formula not checked; hypothesis	6	A
E4, citation slip	R atom; citation gap; summary copy	7.5	A
E5, “zero in area 415”	U atom; scope word; hypothesis	13.5	A
E6, z-scored off level	U atom; hypothesis	7.5	A
E7, exception attached	R atom; exception group; summary copy	6	A
E8, E9, direct vs whole	U atom; ambiguity; hypothesis	13.5	A
E10, E12, E14, double rounding	R atom; boundary; summary copy	4.5	B
E11, hand-summed payload	U atom in a payload	5	A
E15, citation slip	R atom; citation gap	5	A
thal:7 “ONLY”	scope word; hypothesis; verifier refuted	6	G
E13, correct text “corrected”	R atom; lead	3	B

So 12 of the 15 genuine errors land in G or A. The three in B are all double roundings, which is the class that code-filled slots remove (A5). The correct sentence behind E13 stays in B.

5. The independent verifier’s verdict overlay

5.1 Attaching verdicts

`verdict.json` findings already carry `location` (in the skill’s section 6 formats), `part`, `claims` and `statement`. Each finding is joined to a sentence or claim in four steps:

1. Map the location to its text field, e.g. `drafted.readdown.ann_12.evidence[0]` or `operations.<name>.hypothesis`.
2. Within that field, pick the sentence that contains the finding’s `statement`, by fuzzy substring match (token-set ratio of at least 0.8). A finding on a whole field (`record.summary`) attaches to the best-matching sentence, or to the field.
3. Attach `claims.<Cnn>` locations through the claim’s key.
4. Attach findings that match no sentence (structure, coverage, closed forms) to the part or the record. They appear in the queue as part-level items.

One small change to the verifier skill: record `target_hash` in `verdict.json`, the hash of the layer and operations that were verified. The overlay can then tell when a finding refers to text that has since changed, and marks it **stale**.

Display. Each sentence shows a chip with the worst grade and its level, e.g. “refuted L1”, “weakened L2” or “flag L4”. With two verifiers, both chips appear, so agreement is visible.

5.2 Adjudicating a disagreement

A disagreement is any of these:

- the verifier refuted or weakened a sentence that is C or that the human confirmed;
- the verifier upheld a sentence that the human marked wrong;
- two verifiers disagree.

The adjudication form is attached to the finding:

Field	Values
Outcome	`verifier right` (the text is wrong; fix it); `verifier wrong`; `partly` (e.g. right number, wrong label: E8, E9); `cannot tell`
If the verifier is wrong, why	a category from the review's false-positive table: wrong population, wrong referent, wrong quantity, another configuration, notation, idiom, rounding, tool bug
The deciding check	a command and its output (required for every outcome except `cannot tell`)
Action	none; amend (`amend_evidence` or re-render); erratum; reword the scope

Three rules apply:

- **The errata rule from P0.** "Verifier right" for a numeric finding needs the old value shown to be wrong under every reasonable reading: population, scope, split and rounding. This is what would have stopped E13.
- **Feedback into the benchmark.** Every adjudication is also a labelled example, which lets the verifier's detection and false-positive rates be updated beyond run 1. Run 1's ADJUDICATION table is the prototype.
- **Effect on the sentence's verdict.** The adjudication writes a verdict on the sentence (section 6): `verifier right` writes `wrong`; `verifier wrong` writes `confirmed`, with the note. The two records therefore cannot drift apart.

6. Sentence- and claim-level human verdicts that survive re-drafts

6.1 Stable keys

Target	Identity key (stable)	Content hash (changes when what was verified changes)
Sentence with references (A2)	`sid` = sha1(location kind + part id + normalised **source** text with the references kept as written)[:10]	sha1(rendered text + the referenced values at full precision + the hashes of the cited claims)
Sentence with no references (legacy)	`sid` = sha1(location kind + part id + text with **numbers masked** as `#`)[:10]	sha1(rendered text)
Claim	the claim `key` (e.g. `ann_31:direction:age`)	sha1 of its stated fields (`risk`, `n`, `shape.class`, `points`, `groups`)
Query (A1)	the query id plus the hash of its tool and arguments	sha1(values)
Domain question	dataset + `flag` text (shared across genomes)	sha1 of the cited claims' content hashes

The **location kind** is the verifier location with the list index dropped, e.g. `drafted.readdown.<part>.evidence`. Dropping the index stops a reordered paragraph from creating new ids. The **part id** is the annotation id (`ann_N`), so renaming an annotation keeps the key.

A sentence that has no exact `sid` match is **matched by fuzzy text** within the same location kind and part (token-set ratio of at least 0.8). It is then shown as "edited", with the old verdict attached but **stale**. Fuzzy matching is a nice-to-have; without it, an edited sentence simply shows as new.

6.2 The verdict store

Verdicts go in `experiments/<study>/review/<label>.verdicts.jsonl`. The file is append-only, tracked in git, and separate from the database, the record and the layer. That means:

- re-extracting claims, re-drafting, errata and `build_explanations_csv.py` never touch it;
- the export reads it.

One line per verdict:

```
{"id": "v-000123", "target": {"kind": "sentence", "sid": "s-9f2c41d0",
  "location": "operations.ischaemia_pathway.hypothesis[2]", "content_hash": "c-77ab.."},
 "verdict": "confirmed", "note": "checked q4's where-expression against the gate", "by": "mike",
 "at": "2026-11-14T10:05:00+13:00", "seconds": 40,
 "context": {"layer_claims_hash": "...", "ops_hash": "...", "code": "..."},
 "supersedes": null, "source": "ui|cli|adjudication|sample"}
```

- **Verdict values:** `confirmed`, `wrong` (the note says what is wrong; it creates a to-fix item) and `unsure` (the note says why).
- **Latest wins:** the latest line for a target is its current verdict, and nothing is ever deleted.
- **Verdicts on claims** do not stand in for verdicts on sentences. Confirming a claim confirms that its numbers describe the model; each sentence that cites it still needs its own wording checked. The claim verdict is shown next to those sentences.
- **The genome-level `mike_verification` block** stays in the record (A6 compatibility). Sign-off (section 7) writes it.

6.3 "Changed since you last verified"

For each target with a verdict, the current state is one of:

- **unchanged:** the content hash is the same;
- **changed:** same `sid`, different content hash;
- **edited:** fuzzy match only;
- **removed:** the target no longer exists.

Targets without a verdict are **new**. Changed and edited targets are gates (section 4.1).

The diff view shows, per changed item:

- the old and new rendered text, with a word diff;
- each reference whose value changed, old and new;
- **why** it changed, from:
  - `amend_evidence` history (A4);
  - errata entries in the record;
  - claim re-extraction (`claims_hash`);
  - operation changes (`ops_hash`);
  - a re-draft (`drafted_at`).

Example: E10 after the v5 re-draft would have shown "about 4%" become "about 5%", with no change in C086's value (0.04455). That is a red flag that needs no checker.

CLI: `explaneat review diff <label> [--since <date>|--verdicts]`. In the PDF, the verification pack gets a "Changed since your last review" section before the queue, and leaves it out when nothing has changed.

7. Statistical sign-off and a defined "Mike-verified"

7.1 The protocol

The items of one genome are split into strata: G (gates), A (top 25), A' (tier-A overflow) and BC (tiers B and C).

1. **Gates:** every G item gets a verdict. No `wrong` may remain unfixed. Fixing means amending, re-rendering and confirming again, or recording an erratum.
2. **Tier A:** every A item gets a verdict.
3. **Sample:** code draws 10 items from BC, and 10 from A' if A' is not empty. The draw is uniform at random, seeded by sha256(label + content hash of the whole report). The reviewer does not choose the items, and re-running the draw on the same report gives the same items. The sample is labelled `source: sample` in the verdict store.
4. **Sequential rule:** if the sample contains a `wrong`, draw 10 more from the same stratum. If the second draw also contains a `wrong`, the genome is **not verified**. The whole stratum must then be reviewed, or the text regenerated with the integrity tools.
5. **Reproduce:** a full reproduce run (section 2.2) with no mismatch, at the current content hash.
6. **Unsure verdicts:** an `unsure` in G or A is allowed only if the sentence is disclosed. Either the rendered text carries a mark, e.g. "(not independently confirmed)", or the sentence is listed in the report's "Statements not confirmed" appendix.
7. **Domain questions** (section 8.1) are answered or explicitly deferred. They do **not** block verification: they judge the model, not the explanation's faithfulness to it.
8. **Time:** minutes are recorded from the verdict timestamps, or from the UI timer, which already exists.

`explaneat review signoff <label>` checks steps 1 to 6. It writes:

```
"mike_verification": {"verified": true, "minutes": 22, "changes": "...", "notes": "...",
  "protocol": "signoff-v1", "at": "...", "content_hash": "...", "verdicts_hash": "...",
  "strata": {"G": [5, 5], "A": [14, 14], "A_prime": [0, 0], "BC": [137, 10]},
  "sample_wrong": 0, "wrong_fixed": 2, "unsure_disclosed": 1, "reproduce": "all match"}
```

Each stratum entry is [items, reviewed]. The existing keys `verified`, `minutes`, `changes` and `notes` keep their meaning, so `build_explanations_csv.py` and the Review page keep working.

7.2 A bound on the residual error rate

Write CP+(x, n) for the one-sided 95% Clopper-Pearson upper bound, and CP-(x, n) for the lower bound.

Unreviewed stratum (BC, and A'). Pool the samples across genomes: N_BC items in total, n sampled, x found wrong. Then:

```
p_BC <= CP+(x, n)                    (error rate per sentence in the unreviewed stratum)
```

If Mike's own detection rate d_M has been measured (section 7.3), the bound becomes CP+(x, n) / CP-(found, seeded).

Reviewed strata (G and A). Each item there was read by Mike and by a blind verifier. Suppose k errors were found and fixed. If the two readers miss errors independently, with miss rates m_V = 1 - d_V and m_M = 1 - d_M, then the expected number of errors that remain is:

```
residual_GA <= k * (m_V * m_M) / (1 - m_V * m_M)
```

- **The conservative reading** counts no credit for Mike (m_M = 1): residual_GA <= k (1 - d_V) / d_V.
- **The independence assumption** is optimistic, because hard errors are hard for both readers. That is why the conservative reading is the headline figure.

The verifier's detection rates from run 1, as 95% lower bounds:

Fault set	Detected	Rate	95% lower bound
Injected L1 (mechanical)	16 / 16	1.00	0.83
Injected L2 (evidential), strict	12 / 17	0.71	0.48
Injected L3 (adversarial)	13 / 13	1.00	0.79
Injected L4 (domain), strict	1 / 4	0.25	0.01
All injected, strict	42 / 50	0.84	0.73
Natural errors in the 12 benchmark bases	9 / 12	0.75	0.47

The last row counts E1 to E11 plus thal:7. E3, E6 and E9 were not found blind; E6 and E9 were found only while confirming a neighbouring error. So **natural errors are found less often than injected ones**, and the thesis should use 0.47 as d_V's lower bound.

Worked example. Take 30 genomes. Each contributes 10 BC items to the sample, so n = 300 from N_BC of about 4,500.

Sample result	p_BC bound	Expected residual errors in BC (all 30 genomes)
x = 0	1.0%	at most 45, i.e. 1.5 per genome
x = 1	1.6%	at most 71
x = 2	2.1%	at most 94

For the reviewed strata, suppose k = 20 errors were found and fixed:

- with d_V = 0.47 and no credit for Mike, the residual is at most 22.6;
- with d_M = 0.5 measured and independence assumed, it is at most 20 x 0.265 / 0.735 = 7.2.

These bounds are wide on purpose. They are per sentence and honest, and they shrink as provenance moves sentences from U to C. **Report them per sentence, with the strata sizes, never as "the explanations are correct".**

Suggested thesis sentence: "Of N sentences in the M Mike-verified explanations, every gated or high-risk sentence (n_GA) was checked by the author and by a blind verifier (k errors found and fixed). A seeded random sample of n of the remaining N_BC found x errors, so at most p_BC of those are wrong (one-sided 95% Clopper-Pearson)."

7.3 Optional calibration: Mike's detection rate

The verifier benchmark's private builder can inject faults. Put 2 seeded faults into each sign-off batch of about 5 genomes, at random G, A or sample positions, rendered like any other sentence. They are revealed and removed after the batch's verdicts are in, and before anything is written to the records. This measures d_M, as CP-(found, seeded), which the bound in section 7.2 uses. It costs about 2 extra minutes per batch. It is a nice-to-have: without it, the thesis reports the conservative bound.

7.4 Defining "Mike-verified" for the thesis

Definition (`signoff-v1`). A genome is **Mike-verified** (`verified_by_mike = yes`) when four things hold:

1. The automated checks agree (the verification pack's "Recorded vs live" table has no DISAGREE).
2. `review signoff` passes steps 1 to 6 of section 7.1.

3. The record's `mike_verification.protocol` is `signoff-v1`, and its `content_hash` equals the current report's hash.
4. No change to the report has happened since then. Otherwise the genome shows as "verified, stale" and counts as **not verified** until the changed items are re-verified.

Compatibility with the pre-registration:

- PREREG.md lists `verified_by_mike` as a recorded field and requires that agent-only and Mike-verified results are reported in separate columns, never merged. It does not define the field. No genome is verified yet: the 10-seed README says that `verified_by_mike` is "no" for every network.
- Fixing the definition now is therefore a pre-specification, not a post-hoc change. Add it as a dated PREREG amendment ("2026-11-xx: definition of `verified_by_mike` = signoff-v1"), before the first verification.
- `build_explanations_csv.py` keeps `verified_by_mike` (yes/no) and adds six columns:
  - `mike_protocol`, `mike_items_reviewed`;
  - `mike_sample_n`, `mike_sample_wrong`;
  - `mike_wrong_fixed`, `mike_unsure_disclosed`.
- A genome that Mike verified with errors fixed counts in the "Mike-verified" column with its corrected text. The agent-only column keeps the original outcome, as the pre-registration requires.

8. Domain questions, UI, and the changes needed

8.1 Domain questions ("worth checking")

Each `worth_checking` item becomes a **domain question**. It is keyed by dataset and its `flag` text, which is already reused across genomes, e.g. "high fasting blood sugar lowers risk". It is shown with:

- the sentence;
- the cited claims' numbers, n and test check;
- the genomes of the same dataset that raise the same flag, from the existing `plain-language worth-checking` table.

Answers are made once per dataset and apply to every genome with that flag. A genome-specific override is possible. One line per answer goes in `experiments/<study>/review/domain_answers.jsonl`:

Field	Values
answer	`plausible, known` (with a reference); `plausible, new to me`; `implausible: likely a data or model artefact`; `needs a data check`; `outside my field`
reference	a citation or guideline, optional
note	free text
by, at	e.g. Mike, or the clinical collaborator's name

Domain answers judge the **model**, not the explanation. An "implausible" answer is a finding about what the network learned, and it belongs in the thesis discussion. It does not make the explanation wrong. A collaborator gets only the questions: a 2 to 3 page PDF per dataset, with tick boxes and room for notes, which Mike enters with `explaneat review domain-answer`. A web form for collaborators is a nice-to-have.

8.2 UI changes

Component	Change	Priority
`RecordsPage` (`/review`)	New columns: badge counts (C/R/U/F), queue size (G/A), sign-off state (none, in progress, verified, verified-stale), changed-since-verified count. A filter "needs attention".	must (S)
New `StatementsPanel` (a ToolsDrawer tab "Statements")	A **queue mode** listing G and A items, then the sample. Each row shows the badge, the risk factors, the verifier chip, and buttons for confirmed, wrong and unsure, with a note. Keyboard: j/k to move, 1/2/3 to give a verdict.	must (M, minimal)
`StatementsPanel`, reading mode	The read-down document with badges, linked numbers, provenance cards, and Reproduce.	nice (M)
`StatementsPanel`, diff	"Changed since you verified": word diff and reason, per item.	must, as a list (S); nice, inline
`RecordPanel`	The verdict form gains a sign-off checklist: steps 1 to 6 with live state, a "Draw sample" button and "Sign off" (calls `review signoff`). The existing fields stay.	must (S)
Domain questions tab	A list per dataset with the answer form.	nice (S)
`api/tools.ts` and `explaneat/api/routes/records.py`	Routes: `GET /api/records/{label}/statements`, `GET .../queue`, `POST .../verdicts`, `GET .../diff`, `POST .../reproduce`, `POST .../signoff`, `GET/POST /api/domain-questions`. File writes only; no database.	must (S)

8.3 Changes to schemas, the pack and the CLI, with effort and order

Effort: S is half a day or less; M is 1 to 2 days. "IB" marks items that depend on the integrity branch.

#	Change	Where	Priority	Effort
1	Sentence splitter, `sid` and content hash, number atoms	new `explaneat/review/sentences.py`	must	S
2	Verdict store (jsonl), `review verdict`, `review list`	`explaneat/review/verdicts.py`, CLI	must	S
3	Status badges from the `recheck` report (C/R/U/F; C needs IB refs)	`explaneat/review/status.py`	must	S
4	Risk factors, score, tiers, queue; queue table in the verification pack, replacing `what_to_look_at`	`review/queue.py`, `export/verification.py`	must	M
5	Verifier overlay join and adjudication form; `target_hash` in the verifier skill's output	`review/overlay.py`, skill	must	S
6	Diff since last verdict (CLI and pack section)	`review/diff.py`, `export/verification.py`	must	S
7	Sign-off: sampling, sequential rule, `mike_verification` fields, CSV columns, PREREG amendment	`review/signoff.py`, `build_explanations_csv.py`, `PREREG.md`	must	S
8	Minimal Statements tab (queue, verdict buttons, diff list), RecordsPage columns, sign-off checklist, API routes	UI, `routes/records.py`	must	M
9	Provenance cards: JSON section (schema 1.3), PDF superscripts and appendix, `review reproduce`	`export/`, `review/reproduce.py`	must (IB)	M
10	Domain questions: answers file, CLI, collaborator PDF	`review/domain.py`, `export/render_plain.py`	must-light	S
11	Reading mode with provenance popovers and Reproduce in the UI	UI	nice	M
12	Fuzzy sentence matching ("edited")	`review/sentences.py`	nice	S
13	Seeded faults for d_M, using the benchmark builder	`review/calibrate.py`	nice	S
14	Domain-question web form	UI	nice	S

Implementation order:

1. Items 1, 2, 3 and 7, about 1.5 days. These are enough to sign off a genome from the CLI and the PDF, and they need nothing from the integrity branch.
2. Items 4, 5 and 6, about 2 days. The pack becomes the review instrument.
3. Item 8, about 1.5 days. Recording verdicts becomes fast.
4. Item 9, about 1 to 1.5 days, after the integrity branch merges.
5. Item 10, half a day.

That totals about 6 to 7 days for the must-haves. The nice-to-haves (11 to 14) add about 4 days.

If the integrity branch differs from A1 to A7:

- Only items 3 (the C badge), 6 (the reasons in the diff) and 9 change.
- Items 1, 2, 4, 5, 7 and 8 work on legacy text through `recheck`.
- The sign-off is the same; only the strata sizes differ.

Keeping the process light:

- The pack, the CLI and one UI tab are the whole surface.
- Verdicts are one line each, and no new database tables are needed.
- The review time per genome is bounded by the 25-item cap plus 10 sampled items. At about 30 to 45 s per item, that is 15 to 25 minutes, or 25 to 45 on legacy records.